

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

***CO2:** Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 3

Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I Vempati Murale Karrtik (192312567) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

V Murale Karrtik (192312567)

Signature of the student:

SCENARIO 1: AI-BASED WEDDING SEATING ARRANGEMENT USING CSP

1. Scenario Understanding

An event management company wants to develop an AI system that automatically generates seating arrangements for a large wedding banquet. The system must satisfy several requirements simultaneously:

- Guests with known conflicts must not sit at the same table.
- Each table has limited seating capacity.
- Certain family groups must remain together.
- Guests with dietary requirements need suitable tables.
- Guests with accessibility requirements need appropriate locations.
- VIP guests should be seated near the head table.

As the number of guests and tables increases, the number of possible arrangements becomes extremely large. Therefore, this problem can be formulated as a Constraint Satisfaction Problem (CSP).

2. CSP Formulation

A CSP is represented as:

$$\text{CSP} = (X, D, C)$$

where:

- X = set of variables
- D = domains of possible values
- C = constraints

Variables

Each guest is represented by a variable:

$$X = \{G1, G2, \dots, G_n\}$$

For example, $G1$ represents Guest 1.

Domains

The domain contains the tables where a guest can potentially sit.

For five tables:

$$D(G_i) = \{T1, T2, T3, T4, T5\}$$

Domain values can be reduced based on requirements.

For example, if a wheelchair-accessible guest can use only Tables 1 and 2:

$$D(G_i) = \{T1, T2\}$$

Constraints

Constraint	Meaning
Conflict	Conflicting guests cannot sit at the same table
Capacity	A table cannot exceed its seat limit
Family	Required family members should sit together
Dietary	Guests need tables supporting their dietary requirements
Accessibility	Guests must be assigned to suitable locations
VIP	VIP guests should be close to the head table

For conflicting guests, A and B:

$$\text{Table}(A) \neq \text{Table}(B)$$

For family members A, B and C:

$$\text{Table}(A) = \text{Table}(B) = \text{Table}(C)$$

Thus, the AI searches for an assignment satisfying all hard constraints.

3. Backtracking Search

Backtracking assigns guests to tables one at a time and checks whether each assignment is consistent with the constraints.

Consider:

- A and B cannot sit together.
- C and D must sit together.

- Each table has two seats.

First:

[A = T1]

If the algorithm tries:

[B = T1]

the assignment is rejected because A and B conflict.

The algorithm backtracks and tries:

[B = T2]

For the family group:

[C = T3, D = T3]

provided the table has sufficient capacity.

The process is:

Select guest



Select possible table



Check constraints



Valid?

↓ Yes ↓ No

Assign Backtrack



Continue



Complete solution

4. Constraint Propagation and Forward Checking

Backtracking can still examine many possibilities. Constraint propagation improves efficiency by eliminating impossible values before they are explored.

For example:

$D(B) = \{T1, T2, T3\}$

If:

$[A = T1]$

and A conflicts with B, forward checking immediately removes T1:

$D(B) = \{T2, T3\}$

Therefore, the algorithm does not waste time trying an impossible assignment.

The Minimum Remaining Values (MRV) heuristic can also be used. The guest with the fewest available tables is selected first. This helps identify failures earlier.

5. Challenges and Scalability

A real wedding seating system may contain hundreds of guests and many interacting constraints. Additional difficulties include:

- Last-minute guest changes.
- Different table capacities.
- Multiple family groups.
- Conflicting dietary and accessibility requirements.
- VIP preferences.
- Hard constraints versus soft preferences.

A brute-force approach becomes impractical because the search space increases rapidly as the number of guests increases. suitable approach is [CSP + Backtracking + Forward Checking + MRV]

Hard constraints should be satisfied first, while soft preferences can be optimized afterward.

Conclusion

CSP is well suited to wedding seating because it represents guests as variables, possible tables as domains, and seating rules as constraints. Backtracking provides systematic search, while forward checking and MRV reduce unnecessary computation. This combination provides a scalable approach for large seating arrangements.

SCENARIO 2: AI AUCTION-BIDDING AGENT USING MINIMAX AND ALPHA-BETA PRUNING

1. Scenario Understanding

A fintech startup is developing an AI agent to compete against other automated bidders in a simulated auction. The AI must make bidding decisions within strict time limits while considering future actions of competing agents.

The problem is adversarial because the opponent is also making decisions and attempting to maximize its own outcome.

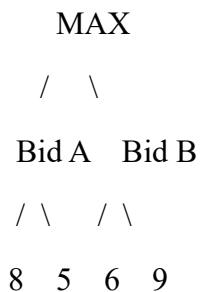
Therefore, Minimax is suitable for modeling the decision process.

2. Minimax Algorithm

Minimax represents the possible bidding decisions as a game tree.

- MAX node: The AI chooses the action giving the highest utility.
- MIN node: The opponent chooses the action that gives the AI the lowest utility.
- Leaf node: Represents a possible future game state.

Example:



For a MAX node:

$$V = \max(V_1, V_2, \dots, V_n)$$

For a MIN node:

$$V = \min(V_1, V_2, \dots, V_n)$$

The AI assumes the opponent will make the most disadvantageous decision for it and therefore selects the best guaranteed option.

3. Computational Challenges

The major problem with Minimax is the size of the game tree.

If:

- b = number of possible actions per state
- d = search depth

then the approximate number of nodes is:

$$O(b^d)$$

For example, if there are 10 possible bidding actions and the AI looks six rounds ahead:

$$10^6 = 1,000,000$$

states may need to be evaluated.

This is expensive when the AI must respond within milliseconds or seconds.

4. Alpha-Beta Pruning

Alpha-Beta pruning improves Minimax by eliminating branches that cannot affect the final decision.

It uses:

- α (alpha) = best value currently available to MAX.
- β (beta) = best value currently available to MIN.

When:

$$\alpha \geq \beta$$

the remaining branch can be ignored.

Example

Suppose the first branch gives MAX a value of 6:

$$\alpha = 6$$

In another MIN branch, the first evaluated value is 5.

Since MIN can already force the result to 5, and MAX already has an alternative producing 6, the remaining values in that MIN branch do not need to be explored.

The important advantage is:

Alpha-Beta pruning gives the same decision as Minimax while potentially examining far fewer nodes.

Good move ordering makes pruning even more effective.

5. Evaluation Function

In a large auction game, searching until the final state may not be possible. Therefore, the AI requires an evaluation function.

A possible function is:

$$E(s) = w_1P + w_2M + w_3R + w_4I - w_5C$$

where:

- P = expected profit
- M = market advantage
- R = remaining budget/resources
- I = information advantage
- C = risk or bidding cost

The weights determine the importance of each factor.

For example:

$$E(s) = 0.4P + 0.2M + 0.2R + 0.1I - 0.1C$$

The weights can be adjusted through simulations and historical game results.

6. Real-Time Performance Strategy

A complete Minimax search is not suitable for a very large auction tree. The AI should combine:

Depth-Limited Search, Search only a fixed number of future rounds, Alpha-Beta Pruning, Eliminate irrelevant branches, Iterative Deepening Search progressively deeper: $d = 1, 2, 3, \dots$

If the time limit is reached, the best result from the last completed depth is used: Move Ordering, Evaluate the most promising bids first to increase pruning, Transposition Tables, Store previously evaluated states to avoid repeating calculations. Recommended Strategy: [Depth-Limited Minimax + Alpha-Beta + Iterative Deepening]

This gives a practical balance between strategic quality and real-time response.

Conclusion

Minimax is appropriate because the auction involves competing intelligent agents. However, its exponential search complexity makes direct implementation expensive. Alpha-Beta pruning, evaluation functions, depth limits and iterative deepening make the AI more suitable for real-time bidding.

SCENARIO 3: AUV SURVIVOR RESCUE USING GREEDY BEST-FIRST SEARCH AND A*

1. Scenario Understanding

A marine research team uses an Autonomous Underwater Vehicle (AUV) to locate survivors near a sunken vessel.

The environment is uncertain and changes continuously:

- Underwater currents change.
- Debris can block paths.
- Visibility varies.
- Movement cost depends on depth and current strength.
- Sonar provides partial real-time updates.
- Rescue requires rapid and safe decisions.

The AUV therefore needs an algorithm capable of finding a route while considering distance, cost and changing conditions.

2. Greedy Best-First Search

Greedy Best-First Search chooses the node that appears closest to the goal.

It uses:

$$f(n) = h(n)$$

where $h(n)$ is the estimated distance from the current node to the survivor.

For example:

Path	Estimated distance
A	100 m
B	150 m
C	200 m

Greedy Best-First Search selects A because:

$$h(A) = 100$$

Advantages

- Simple.
- Fast.
- Uses goal-directed information.
- Can reach a goal quickly when the heuristic is good.

Disadvantages

It ignores the cost already spent.

A path may appear close to the survivor but have:

- Strong currents.
- High energy consumption.
- Dangerous debris.
- Difficult underwater conditions.

Therefore, Greedy Best-First Search does not guarantee the lowest-cost route.

3. A* Search

A* combines the actual cost already travelled with the estimated remaining cost.

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ = actual cost from start to node n
- $h(n)$ = estimated cost from node n to goal
- $f(n)$ = estimated total cost

Example:

Node	$g(n)$	$h(n)$	$f(n)$
A	20	80	100
B	40	50	90
C	10	100	110

A* selects B because:

$$f(B) = 40 + 50 = 90$$

It therefore considers both the distance already travelled and the estimated remaining distance.

4. Variable Underwater Costs

Travel costs are not constant in the AUV environment.

A simplified cost model can be:

$$\text{Cost} = d + \alpha E + \beta C + \gamma R$$

where:

- d = distance
- E = energy consumption
- C = current strength
- R = estimated risk
- α, β, γ = weighting factors

This allows A^* to prefer a route that may be slightly longer but safer and less expensive.

5. Effect of Heuristic Accuracy

The heuristic is important for both algorithms.

Greedy Best-First Search

Greedy search depends strongly on $h(n)$. If the survivor's location changes or sonar information is inaccurate, the algorithm may move toward a path that appears short but is actually blocked or dangerous.

A^*

A^* also depends on $h(n)$, but it additionally considers:

$$f(n) = g(n) + h(n)$$

Therefore, actual travel cost can reduce the effect of a poor estimate.

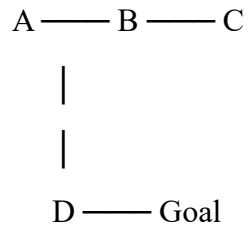
If $h(n)$ is admissible, meaning it does not overestimate the remaining cost, A^* can provide an optimal solution for a static problem under the usual assumptions.

6. Dynamic Environment

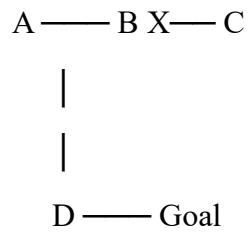
The underwater environment may change while the AUV is searching.

For example:

Initial:



If sonar detects debris blocking B:



The previous route may no longer be usable.

The AUV must therefore:

1. Receive new sonar information.
2. Update the map.
3. Remove blocked paths.
4. Update movement costs.
5. Re-plan the route.

Repeatedly performing a complete search can waste valuable rescue time.

7. Comparison of Greedy Best-First and A*

Feature	Greedy Best-First	A*
Function	$f = h$	$f = g + h$
Previous cost	Ignored	Considered
Heuristic	Required	Required
Speed	Usually faster	Usually slower
Optimality	Not guaranteed	Possible with suitable heuristic

Feature	Greedy Best-First	A*
Variable costs	Poor handling	Better handling
Safety-aware routing	Limited	Better
Dynamic environment	Requires replanning	Requires replanning

8. Recommended Algorithm

For the rescue mission, A* is the better basic algorithm because the AUV must balance speed, travel cost and safety. Greedy Best-First Search focuses mainly on reaching the goal quickly. It may select a dangerous route simply because it appears geographically close.

A* evaluates: Actual Cost + Estimated Remaining Cost Therefore, it is more appropriate when movement costs vary.

However, the environment is dynamic, so the practical system should combine A* with continuous replanning.

*[A + Real-Time Sonar + Dynamic Replanning] **

An incremental replanning algorithm such as D* Lite can also be considered when the environment changes frequently.

Conclusion

A* provides a better balance between optimality, cost and safety than Greedy Best-First Search. For an actual AUV rescue system, however, the algorithm must continuously incorporate sonar updates and re-plan when obstacles or movement costs change.